

Modify copy convert

```
arr.append(7)
arr.reverse()
arr.tolist()
arr.typecode
```

Arrays become useful when you can transform them.

Today: add, reverse, copy, convert, and build from input.

Add items with `append()`



Append

```
from array import array
nums = array('i', [2, 4, 6])
nums.append(8)
print(nums)
```

Adds at end

`append()` places the new value after the last item.

Same type

The value must match the array type code.

Result

`array('i', [2, 4, 6, 8])`

Reverse in place

reverse()

```
nums = array('i', [2, 4, 6, 8])
nums.reverse()
print(nums)
# array('i', [8, 6, 4, 2])
```

In place

The existing array order changes.

No new array

reverse() does not create a separate copy.

Fast check

Print before and after to see the change.

Copying is not assignment



Alias

```
arr1 = array('i', [1, 2, 3])
arr2 = arr1
arr2.append(4)
# arr1 also changes
```



True copy

```
arr1 = array('i', [1, 2, 3])
arr2 = array(arr1.typecode, arr1)
arr2.append(4)
# arr1 stays separate
```

Assignment shares the same array; copying creates independent data.

Convert and reuse type



tolist()

```
nums = array('i', [3, 6, 9])
normal = nums.tolist()
print(normal)
# [3, 6, 9]
```



typecode

```
nums.typecode
# 'i'
copy = array(nums.typecode,
              nums)
```

Use typecode to avoid hardcoding the array data type.

Pythonic array copies



Generator expression

```
nums = array('i', [1, 2, 3, 4])
squares = array(nums.typecode,
                 (n * n for n in nums))
print(squares)
```

No temp list

Values are produced one by one.

Memory aware

Avoids building an intermediate list first.

Same type

typecode keeps the new array consistent.

Cheat Sheet

Python array operations

Practice pattern

```
n = int(input("Size: "))
arr = array('i', [])
for i in range(n):
    arr.append(int(input("Value: ")))
print(arr.tolist())
```

append()

Add a value at the end.

reverse()

Flip the array in place.

copy safely

Use array(arr.typecode, arr).

generator

Build new arrays without an intermediate list.

Save this for your Python journey.

Day 29/55 - Prepared by Satyanarayan Sen